

Introdução a Agentes de IA e Microsoft Agent Framework

Versão .NET 10 LTS / C# 14 — 3 horas

Agenda da aula

Recap RAG: o que aprendemos sobre qualidade

O que é um agente de IA?

Padrões: ReAct, planning, tool use

Microsoft Agent Framework — visão geral

Hands-on: primeiro agente

Recap

Conceito

Padrões

MAF

Hands-on

Recap em 1 minuto — RAG

Pipeline retrieve+generate consolidado

Métricas de qualidade são obrigatórias

Hybrid + semantic ranking são ganho fácil

Próximo nível: ação, não apenas resposta

De responder para agir

- RAG: aprendi a responder
- Agente: vou aprender a agir
- RAG é uma das ferramentas do agente
- Mudança de paradigma!

Chatbot × RAG × Agente

Agente: decide e age (autonomia)

RAG: recupera e responde

Chatbot: LLM + UI

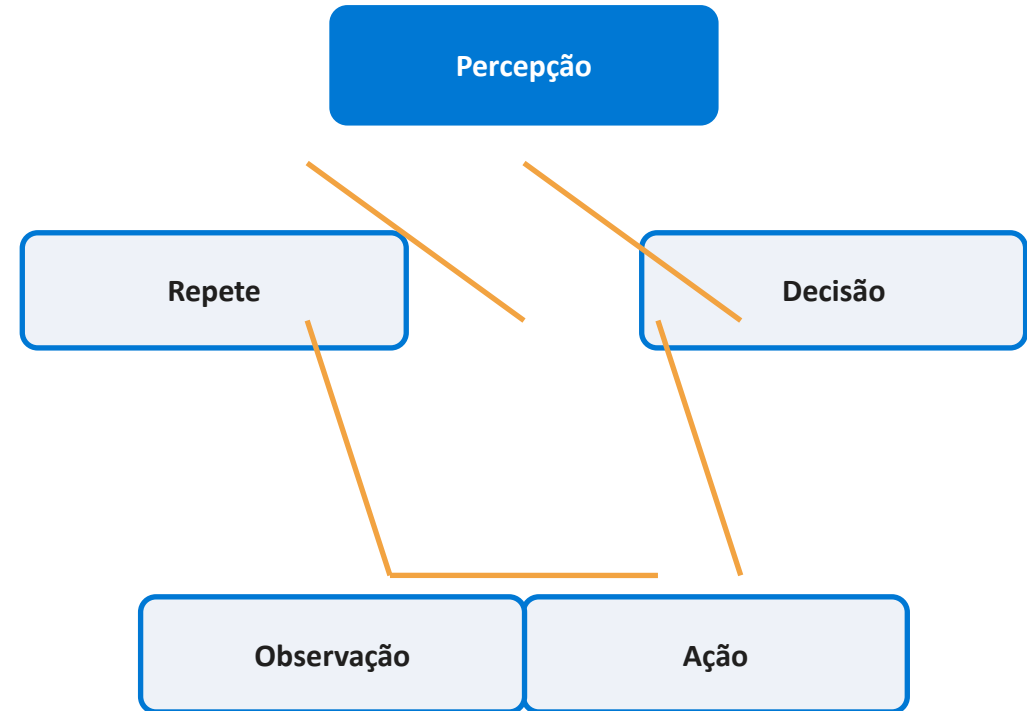
Anatomia de um agente

Modelo (LLM) + Tools + Memória + Planejador

Loop: percepção → decisão → ação → observação

Critério de parada (concluiu? exauriu? errou?)

Identidade (persona) e regras (system message)



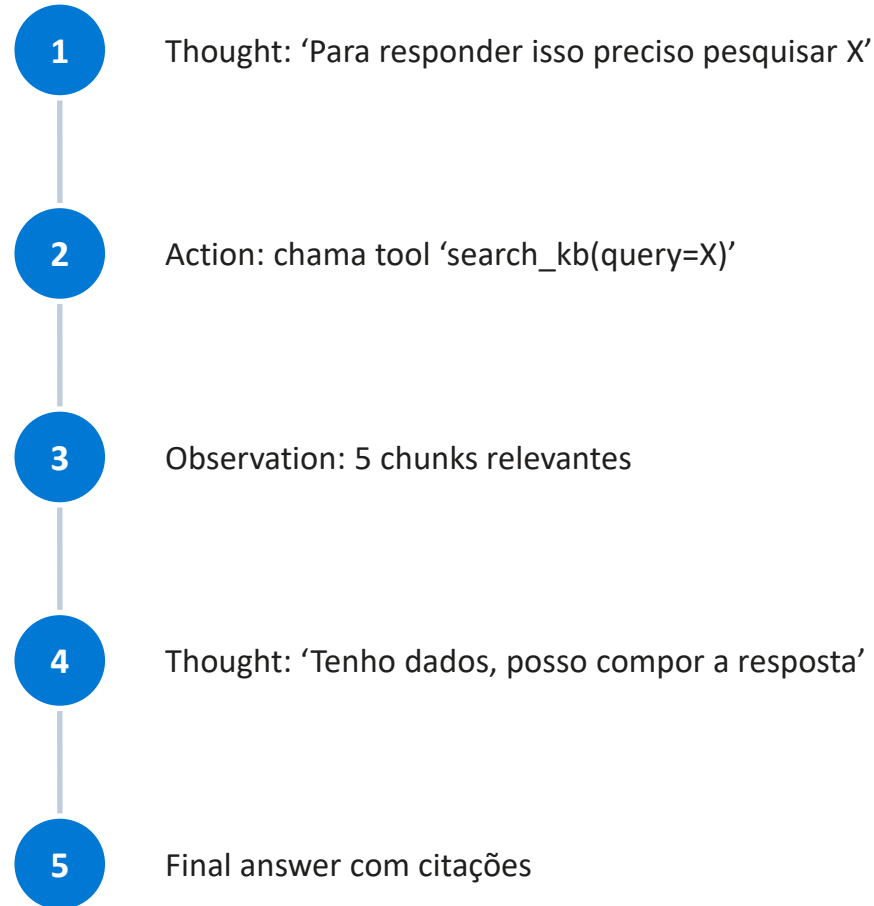
Padrão ReAct

Reasoning + Acting (raciocínio + ações)

Thought → Action → Observation → repete

Padrão simples e amplamente adotado

Funciona bem com modelos premium



Padrão Planning

Plano explícito de N passos antes de executar

Executor segue o plano

Replan em caso de erro

Mais previsível, menos flexível que ReAct



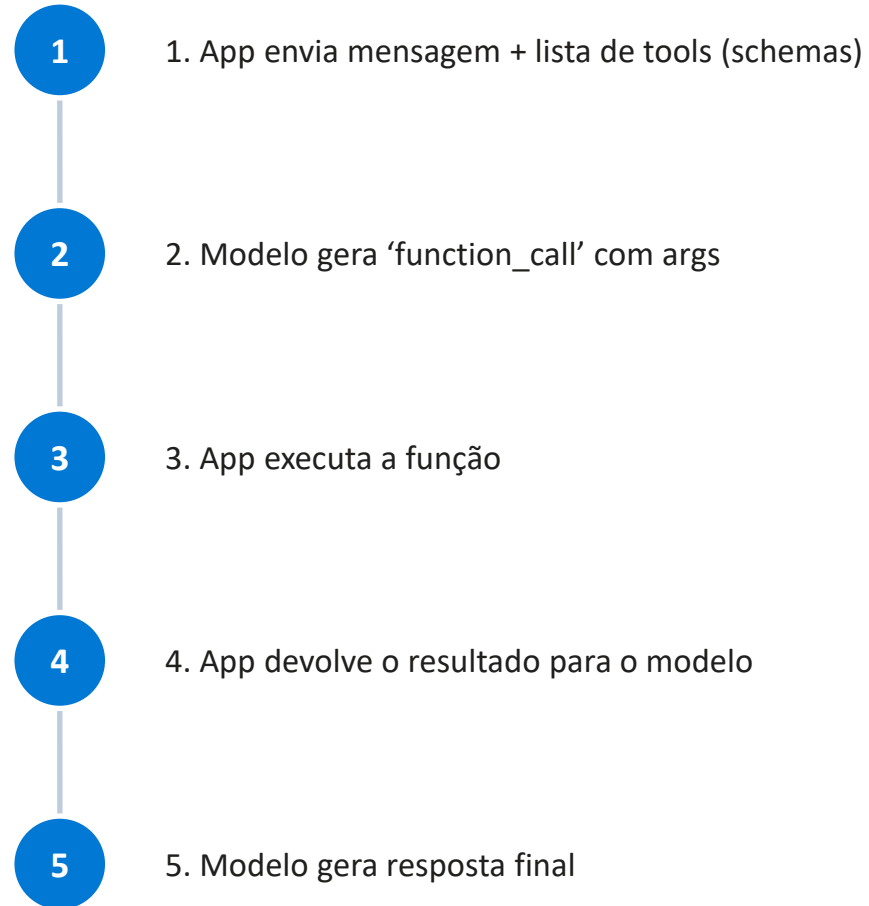
Function calling — o motor das tools

Modelo decide 'chame essa função com esses args'

Aplicação executa e devolve o resultado

Modelo continua a partir do output

Schema declarativo (nome, descrição, parâmetros)



Histórico: Semantic Kernel, AutoGen e o Agent Framework

2023

Semantic Kernel
(skills, plugins)

2024

Convergência
anunciada



2023

AutoGen
(multi-agent)



2026

Microsoft Agent Framework
(1.0 GA)

Capacidades do MAF

Agents

Modelo + tools + agent session
Integração MCP
Streaming, middleware
Para tarefas abertas/conversacionais

Workflows

Grafo de executors
Edges condicionais
Checkpointing + HITL
Para processos definidos

MAF × Foundry Agent Service

MAF (SDK code-first)

Mais flexível
Você gerencia o ciclo de vida
Multi-provider
Para times de engenharia

Foundry Agent Service

Gerenciado (PaaS)
Conversations e sessions gerenciados
Tools nativas (file search, code interpreter, MCP)
Para times mais 'low-code'

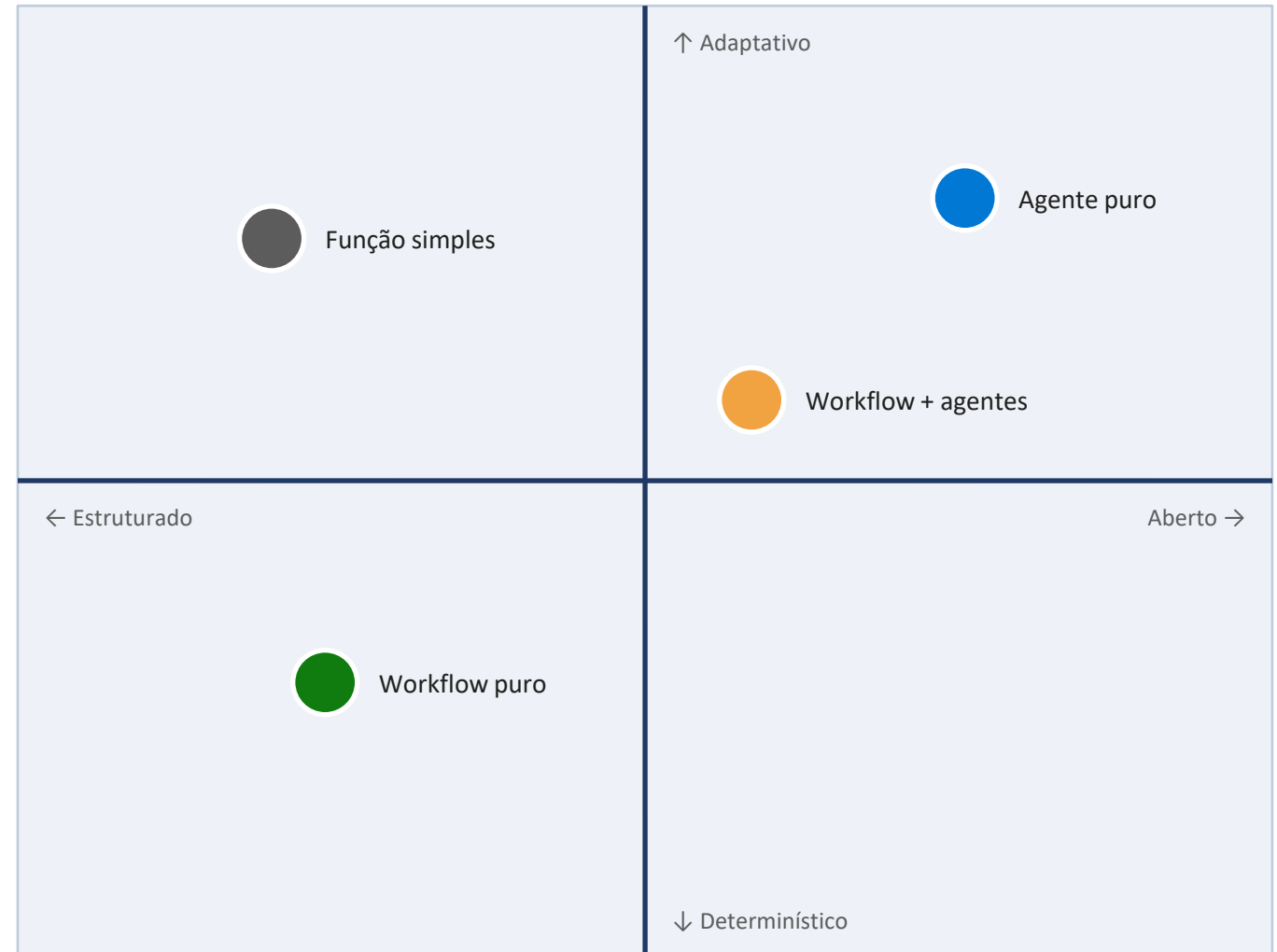
Quando usar agente × workflow

Agente: tarefa aberta, conversacional, exploratória

Workflow: processo bem definido, multi-step, repetível

Workflow pode conter agentes como executores

Regra: 'se uma função simples resolve, use uma função'



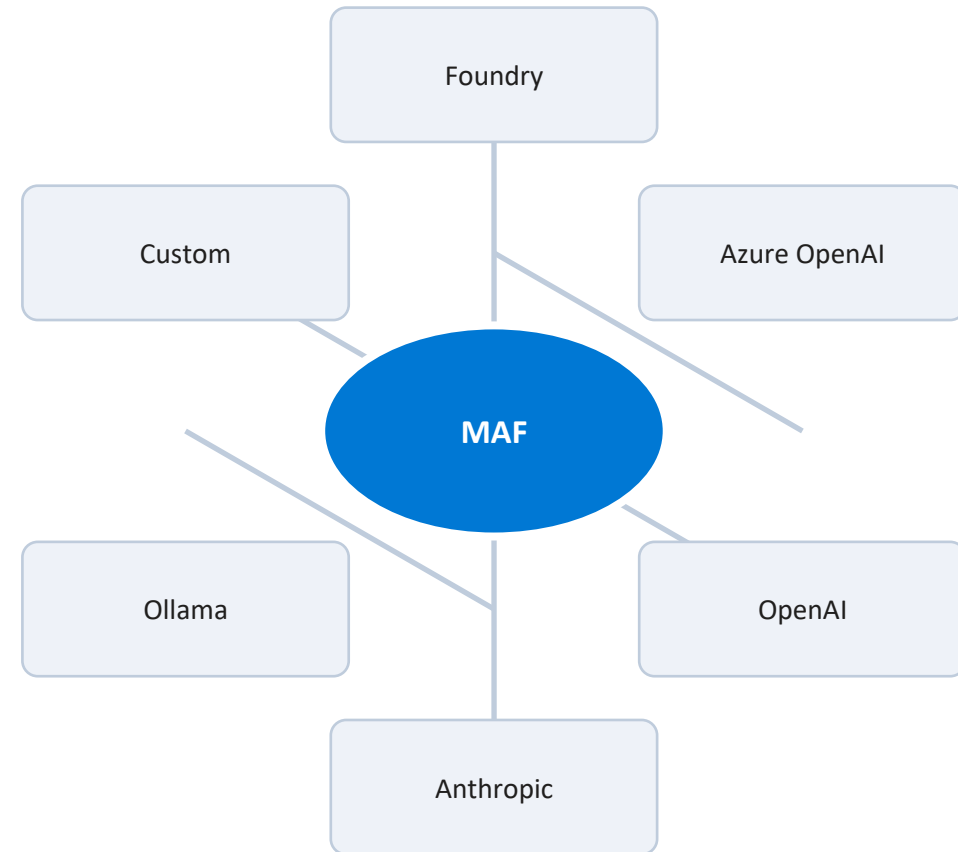
Provedores suportados

Microsoft Foundry, Azure OpenAI, OpenAI

Anthropic, Ollama (local)

Plug-and-play via abstração

Mesma API, diferentes endpoints



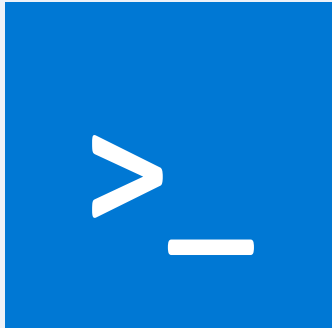
Hands-on: Primeiro agente em C# 14



Hands-on guiado em sala

1. `dotnet add package Microsoft.Agents.AI.Foundry --prerelease`
2. `dotnet add package Azure.Identity (1.21.0)`
3. NÃO declare `Azure.AI.*` nem `Microsoft.Extensions.AI*` — vêm transitivamente
4. `AIProjectClient(...).AsAIAgent(model, name, instructions)`
5. `RunStreamingAsync` para streaming; `AgentSession` para multi-turn

Como construir o projeto da aula



Do zero ao dotnet run

1. `dotnet --version` → precisa responder 10.x
2. `dotnet restore` → `dotnet build` na pasta da solução
3. `copy Properties\launchSettings.template.json Properties\launchSettings.json`
4. Preencher `FOUNDRY_ENDPOINT` e `FOUNDRY_MODEL`
5. `az login --tenant <tenant-id>` → `dotnet run`

Compilou e quebrou em runtime? `dotnet nuget why <csproj> OpenAI`

Um projeto = um pivô do pacote OpenAI. Não misture `Azure.AI.*` com `Microsoft.Extensions.AI.OpenAI`.

Recap da aula

Agente = LLM + Tools + Memória

Loop perceber-decidir-agir

ReAct e Planning

Padrões fundamentais

MAF 1.0 unifica SK + AutoGen

GA desde abril de 2026

Provedores intercambiáveis

Sem lock-in

Quiz da aula



Questionário ao vivo — 5 questões

5 questões sobre agentes e MAF

Discussão das alternativas

Justificativa do gabarito

Quiz — Pergunta 1 de 5



1

O que distingue um agente de um chatbot com RAG?

- A) O agente roda no servidor e o chatbot no cliente
- B) O agente usa um modelo maior
- C) O agente decide, em tempo de execução, quais ferramentas chamar e em que ordem
- D) O agente sempre mantém memória de longo prazo

Quiz — Resposta 1 de 5



C) O agente decide, em tempo de execução, quais ferramentas chamar e em que ordem

Chatbot responde; RAG responde com contexto recuperado; agente age — escolhe e executa ações. Essa autonomia é o ganho e também o risco: menos previsível, mais difícil de testar. Boa parte dos sistemas reais é RAG com uma ou duas tools, e isso basta.

Quiz — Pergunta 2 de 5



No padrão ReAct, o que se alterna a cada ciclo?

- A) Recuperação e avaliação
- B) Prompt e fine-tuning
- C) Treino e inferência
- D) Raciocínio sobre o que fazer e execução de uma ação, com observação do resultado

Quiz — Resposta 2 de 5



D) Raciocínio sobre o que fazer e execução de uma ação, com observação do resultado

Reason + Act: o modelo pensa, chama uma tool, lê o resultado e decide o próximo passo.
O laço termina quando o modelo julga ter a resposta — ou quando você impõe um limite.
Impor limite de iterações não é detalhe: é o que impede o agente de girar indefinidamente.

Quiz — Pergunta 3 de 5



3

Qual pacote você deve declarar para criar um agente Foundry em .NET?

- A) Microsoft.Agents.AI.Foundry (--prerelease) e Azure.Identity — o resto vem transitivamente
- B) Azure.AI.Extensions.OpenAI e OpenAI
- C) Somente Microsoft.Extensions.AI
- D) Azure.AI.Projects e Microsoft.Extensions.AI.OpenAI, na mão

Quiz — Resposta 3 de 5



A) `Microsoft.Agents.AI.Foundry` (`--prerelease`) e `Azure.Identity` — o resto vem transitivamente

Deixe o Agent Framework ser o único dono do grafo: ele já traz `Microsoft.Agents.AI`, `Microsoft.Extensions.AI`, `Microsoft.Extensions.AI.OpenAI` e `Azure.AI.Projects` nas versões que testou. Declarar `Azure.AI.*` à mão neste projeto é exatamente o que recria o `MissingMethodException`.

Quiz — Pergunta 4 de 5



4

MAF (Agent Framework) × Foundry Agent Service: qual a diferença essencial?

- A) MAF é a biblioteca que roda no seu processo; o Agent Service hospeda o agente como recurso do Foundry
- B) O Agent Service não suporta tools
- C) MAF só funciona em Python
- D) São o mesmo produto com nomes diferentes

Quiz — Resposta 4 de 5



A) MAF é a biblioteca que roda no seu processo; o Agent Service hospeda o agente como recurso do Foundry

Com MAF, o agente vive na sua aplicação: você controla deploy, escala e observabilidade.

Com o Agent Service, o agente é um recurso versionado no Foundry, chamável por qualquer cliente.

Dá para combinar: MAF consumindo um agente hospedado.

Quiz — Pergunta 5 de 5



Function calling: quem realmente executa a função C#?

- A) O runtime do .NET, sem passar pelo seu código
- B) O seu processo — o modelo apenas devolve o nome da função e os argumentos
- C) O modelo, internamente
- D) O serviço do Foundry, em sandbox

Quiz — Resposta 5 de 5



B) O seu processo — o modelo apenas devolve o nome da função e os argumentos

O modelo não executa nada: ele emite uma intenção estruturada de chamada.

Quem valida os argumentos e executa é o seu código (ou o pipeline com UseFunctionInvocation).

Por isso a validação de entrada da tool é responsabilidade sua, não do modelo.

Desafio + próxima aula

Agente especialista em .NET 10 (Worker Service + xUnit)

Persona clara, multi-turn, fora-de-escopo, streaming

Documentação de 1 página justificando escolhas

Próxima aula: tools, MCP, memória e multi-agent

Por que

Pratica MAF na sua área

O que

Agente especialista

Quando

24h antes da Aula 7

Próximo

Tools e multi-agent